

TELEDETECTION 03



Auteurs :

CHARRIN Célian

SANTALUCIA Morgan

JOLLY Alexandre

TAULE BONNAIRE Timon

Supervisé par :

PICONE Daniele

GODET Aurélien

Année universitaire 2024 - 2025

Table des matières

1	Introduction	2
2	Objectifs	2
3	Ressources à disposition	3
4	Méthodes exploitables	4
4.1	Traitement de l'image simple	4
4.2	Réseau de neurones personnalisé	5
4.3	Réseau de neurones pré-entraîné	5
5	Détail des méthodes	5
5.1	Cas 1 : Une seule combinaison modèle/backbone pour toutes les classes . .	6
5.2	Cas 2 : Une combinaison modèle/backbone différente par classe	7
6	Analyse de résultats	8
6.1	Résultats du modèle de segmentation totale	8
6.2	Résultat du modèle MOE	11
7	Comparaison des résultats	13
8	Conclusion	13
9	Sources	14

1 Introduction

Ce projet de télédétection a été réalisé dans le cadre du module de parcours à choix, pour mettre en pratique ce qu'on a appris pendant les CTD sur la télédétection et le machine learning. Le but est d'acquérir des connaissances théoriques et de les appliquer à une problématique concrète.

La télédétection, est une technique qui permet d'observer la Terre (ou d'autres objets) à distance, sans avoir besoin d'être sur place (méthode non-intrusive). En général, on utilise des capteurs embarqués sur des satellites ou des drones pour récupérer des images. Ces capteurs peuvent capter différents types d'informations, comme la lumière visible, l'infrarouge, ou même les ondes radar.

Il existe deux grands types de télédétection :

- **Passive** : les capteurs enregistrent la lumière naturelle réfléchiée par les surfaces (comme les images satellites optiques).
- **Active** : les capteurs envoient un signal (comme un laser ou une onde radar) et mesurent ce qui revient après avoir touché la surface.

Pour pouvoir exploiter les images, on utilise des méthodes de classification. Le but est de regrouper les pixels de l'image en différentes classes (forêt, bâtiments, eau, etc.). Il y a plusieurs façons de faire :

- **Classification supervisée** : on apprend à l'algorithme ce que sont les différentes classes à partir d'exemples connus. Exemples d'algorithmes : Random Forest, SVM, réseaux de neurones, etc.
- **Classification non supervisée** : ici, pas besoin d'exemples, l'algorithme trouve tout seul des groupes similaires dans les données (par exemple, avec k-means).

Dans notre cas, on s'est intéressés à la classification supervisée sur la région grenobloise à partir d'images satellites passives RGB.

2 Objectifs

On s'intéresse dans ce rapport à un problème de segmentation urbaine de la ville de Grenoble (Isère 38). Autrement dit, on cherche à diviser une image **RGB** (avec trois composantes pour chaque pixel) en plusieurs classes, chacune correspondant à un objet différent. Dans notre cas d'étude, il y aura huit classes permettant d'identifier des éléments : Bareland, Rangeland, DevelopedSpaces, Road, Trees, Water, Agricultural Land, Buildings. Les classes à segmenter ne sont à priori pas fixées. Il est donc nécessaire de les définir clairement, par exemple en utilisant des jeux de données avec des masques déjà construits (1).

Il est important de noter que la diversité des "objets" présents dans une ville empêche une segmentation parfaite, où toutes les classes seraient identifiées de manière précise. Il

faut donc trouver un équilibre entre la complexité et la pertinence des classes à définir.

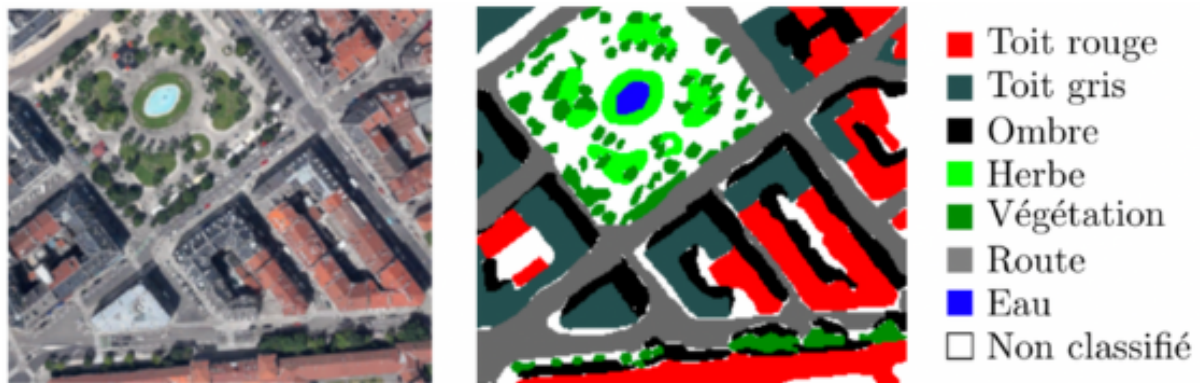


FIGURE 1 – Exemple de masque pour la segmentation urbaine de Grenoble

Il est donc nécessaire d’implémenter plusieurs méthodes en Python pour répondre à ces objectifs. Les principales étapes de l’implémentation incluent :

- *Préparation des données* : Récupérer et prétraiter les jeux de données.
- *Segmentation de l’image* : Appliquer les algorithmes de segmentation sur les images RGB.
- *Analyse des résultats* : Évaluer la précision et ajuster les classes définies.

3 Ressources à disposition

Pour entraîner notre modèle de classification, on commence par utiliser une première base de données mise à notre disposition : une banque d’environ 70 images satellite de Paris, avec leurs masques de classification déjà créés (2). Chaque image est accompagnée d’un label (ou masque) qui indique la classe de chaque pixel (bâtiments, végétation, route, etc.). Cette première base nous permet de faire les premiers tests et d’entraîner un modèle simple.

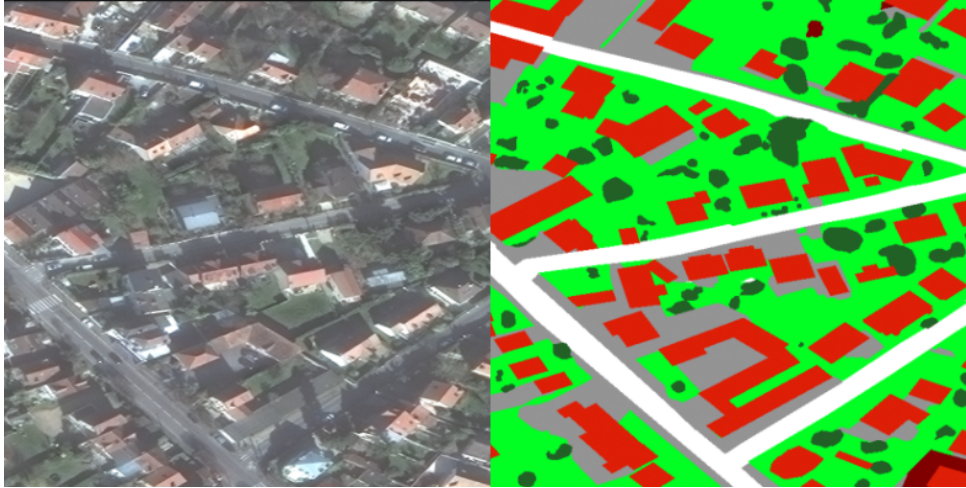


FIGURE 2 – Exemple d'image et masque de Paris

Mais pour éviter que notre modèle soit trop spécialisé sur Paris (surapprentissage ou overfitting), on ajoute une deuxième source de données provenant d'OpenEarthMap. C'est une base de données collaborative, d'où proviennent déjà les images de Paris, qui contient plus de 2500 images satellites annotées. Elle est très utile pour diversifier l'entraînement du modèle, car elle couvre des environnements variés dans le monde entier. Grâce à ça, notre modèle devient plus robuste et plus capable de généraliser, même s'il est utilisé ailleurs que sur Paris ou Grenoble.

Enfin, on crée aussi notre propre petite banque de données. On a utilisé Google Earth pour capturer manuellement des images satellite de la région grenobloise. Google Earth permet d'exporter des images avec une résolution de 650 x 650 pixels. Ces images nous servent à tester le modèle dans une zone qu'on choisit nous-mêmes, et à valider la qualité de nos travaux sur notre région cible.

4 Méthodes exploitables

Pour la bonne réalisation de ce projet, il est nécessaire de chercher à mettre en œuvre différentes méthodes afin d'explorer toutes les possibilités de réponse.

4.1 Traitement de l'image simple

En réalisant des opérations simples de traitement de l'image avec la bibliothèque Python **OpenCV**, le but est d'affiner notre compréhension de la manipulation d'images avec Python, tout en essayant d'implémenter des algorithmes permettant la détection des contours ou des zones de même couleur, par exemple, sur une image donnée.

4.2 Réseau de neurones personnalisé

L'objectif ici est d'utiliser la bibliothèque ***PyTorch*** donnée dans Python pour créer un algorithme de machine learning entraîné sur un dataset comprenant plusieurs images (d'abord des images de Paris avec le dataset donné pour le début du projet, puis avec un dataset comprenant des images de villes du monde entier détaillé précédemment dans le rapport).

Dans cette section, le but est de construire un réseau de neurones personnalisé pour ensuite l'entraîner sur le dataset choisi. La construction du réseau de neurones se fait par le choix d'un encodeur/décodeur. L'encodeur permet de diviser l'image initiale en plusieurs images par convolutions successives (pour permettre à l'IA de détecter des patterns) puis le décodeur permet de comprendre ces images pour associer une classe à chaque pixel, aussi par convolutions successives.

4.3 Réseau de neurones pré-entraîné

- *Modèle pour la segmentation totale* : Avec ce modèle, l'idée est de créer un modèle polyvalent sur toutes les classes disponibles capable de classifier complètement une image. Le détail de cette méthode est disponible dans la partie 5.
- *Modèle MOE (Mixture Of Experts)* : Avec ce modèle, l'objectif est d'entraîner huit algorithmes différents, chacun capable d'identifier la présence d'une des classes fournies. On construit ainsi des "experts" pour chaque classe qu'il va falloir ensuite superposer par une étude probabiliste. Le détail de cette méthode est disponible dans la partie 5.

5 Détail des méthodes

Pour la segmentation des images satellites, nous testons deux approches différentes, chacune reposant sur l'utilisation de combinaisons modèle/backbone. Nous développons un programme (voir notebook associé) permettant de modifier facilement le backbone, le modèle utilisé, ainsi que le nombre d'époques d'entraînement. Le programme renvoie un masque d'image, où chaque pixel est classé dans l'une des classes cibles.

Le backbone correspond à l'architecture de base d'un réseau de neurones convolutif. Il est responsable de l'extraction des caractéristiques à partir de l'image (features). Des exemples connus incluent ResNet, EfficientNet, ou MitBX (2).

Backbone	ResNetX	EfficientNetBX	MitBX
Avantages	Très bon pour l'extraction de textures complexes à différentes échelles	Efficace sur des jeux de données réduits, bon compromis complexité/performance	Adapté à la segmentation urbaine sur de très grandes zones, précis sur les formes géométriques

TABLE 1 – Comparaison des backbones testés

Le modèle, quant à lui, correspond à la structure globale de segmentation (comme U-Net, DeepLabV3, etc.), qui utilise le backbone pour produire une prédiction par pixel (sous forme de masque) (1).

Modèle	FPN	U-Net	U-Net++	DeepLabV3+
Avantages	Capture bien les détails fins et structures complexes	Très performant sur petits jeux de données, bonne extraction de détails	Version améliorée de U-Net, meilleure segmentation des contours	Excellente performance sur images haute résolution

TABLE 2 – Comparaison des modèles de segmentation testés

5.1 Cas 1 : Une seule combinaison modèle/backbone pour toutes les classes

Dans ce premier scénario, nous utilisons une seule combinaison modèle/backbone pour classer simultanément les 8 classes définies dans notre projet (bâtiments, routes, végétation, etc.).

Le modèle est entraîné sur l'ensemble de la banque de données. Le critère d'arrêt est basé sur la stagnation de l'IoU et de la validation loss pendant 20 époques.

Une époque correspond à un passage complet du modèle sur l'intégralité de l'ensemble d'entraînement. Autrement dit, chaque image de la base est vue une fois par le modèle durant une époque. En général, plusieurs époques sont nécessaires pour que le modèle apprenne correctement les motifs et les régularités présentes dans les données.

- **IoU (Intersection over Union)** : mesure la qualité de la prédiction en comparant la zone prédite à la vérité terrain. Plus l'IoU est proche de 1, meilleure est la prédiction.

- **Validation loss (Val_loss)** : indique à quel point le modèle se trompe sur les données de validation. Une faible valeur est préférable.

À chaque époque, le programme enregistre le meilleur modèle selon un score pondéré :

$$\text{Score} = 0.6 \times \text{IoU} + 0.4 \times (1 - \text{val_loss})$$

Nous choisissons de donner plus d'importance à l'IoU car, d'après la documentation des modèles testés, cette métrique reflète mieux la qualité des prédictions en segmentation. Voici un exemple d'évolution de ces paramètres lors de l'entraînement d'un modèle (3).

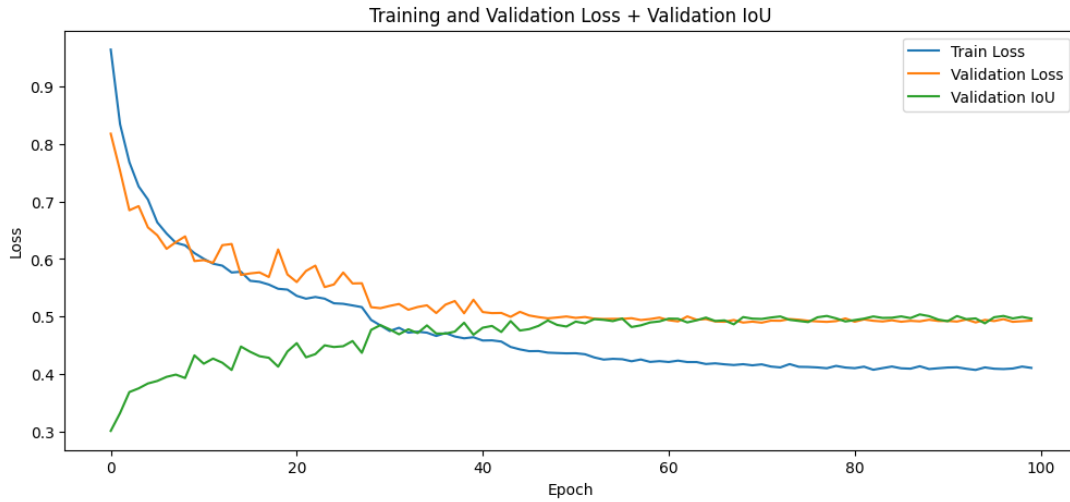


FIGURE 3 – Evolution du IOU et de la validation loss lors de l'entraînement d'un modèle

À l'issue de l'entraînement, nous comparons visuellement les résultats sur un échantillon de la banque de test pour évaluer les performances. Plusieurs combinaisons de modèle et de backbone sont testées afin d'identifier celle qui donne les meilleurs résultats globaux.

5.2 Cas 2 : Une combinaison modèle/backbone différente par classe

Dans ce second scénario, l'idée est d'entraîner un modèle spécialisé pour chaque classe. Le choix du modèle et du backbone est fait en fonction de la documentation et des propriétés spécifiques à chaque classe (capacité à détecter des contours, lignes droites, zones homogènes, etc.). Cette approche est probablement plus efficace, car chaque modèle est optimisé pour une seule tâche (ex. : détecter uniquement les bâtiments ou uniquement la végétation).

Le principe d'entraînement reste le même que dans le cas précédent. La différence principale est que le programme ne renvoie plus un masque global, mais une carte de probabilités

pour chaque classe. Cela signifie que pour chaque pixel, on obtient un score entre 0 et 1 indiquant la probabilité d'appartenir à la classe considérée (4).

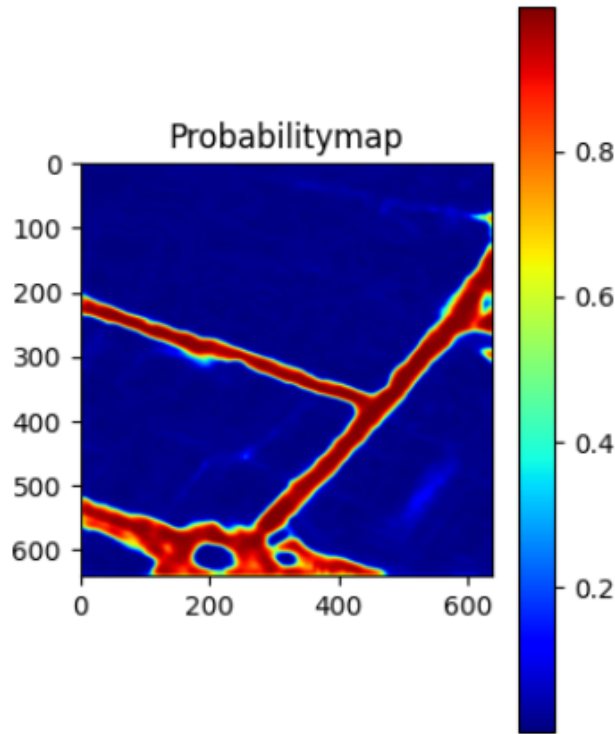


FIGURE 4 – Carte des probabilités de "l'expert route" sur une image

Dans un premier temps, nous attribuons à chaque pixel la classe avec la probabilité la plus élevée. Cependant, nous observons un problème : le modèle entraîné pour les bâtiments produisait des probabilités excessivement élevées, ce qui entraîne une surclassification (des bâtiments étaient détectés partout). Pour corriger cela, nous introduisons un seuil critique, en dessous duquel un pixel n'est pas considéré comme appartenant à la classe bâtiment, même si sa probabilité est la plus haute parmi les autres classes. Cela permet de filtrer les fausses détections et d'améliorer la précision globale.

6 Analyse de résultats

6.1 Résultats du modèle de segmentation totale

Pour ce modèle, on a utilisé la combinaison de modèle Unet avec backbone efficientnet-b5 sur les données complètes du dataset Openearth-Map. Cette configuration a été choisie car elle permet le meilleur rapport qualité / temps d'exécution avec les ressources à notre disposition (55 minutes par époques sur 35 époques). On obtient alors un modèle capable de nous rendre ce type de masque :

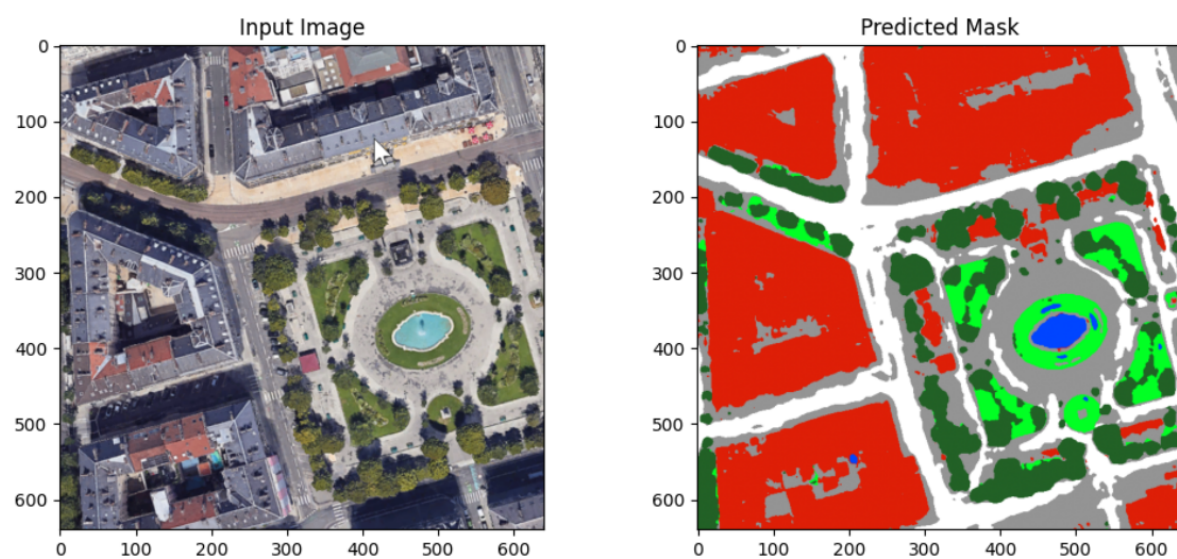


FIGURE 5 – Exemple de résultat 1

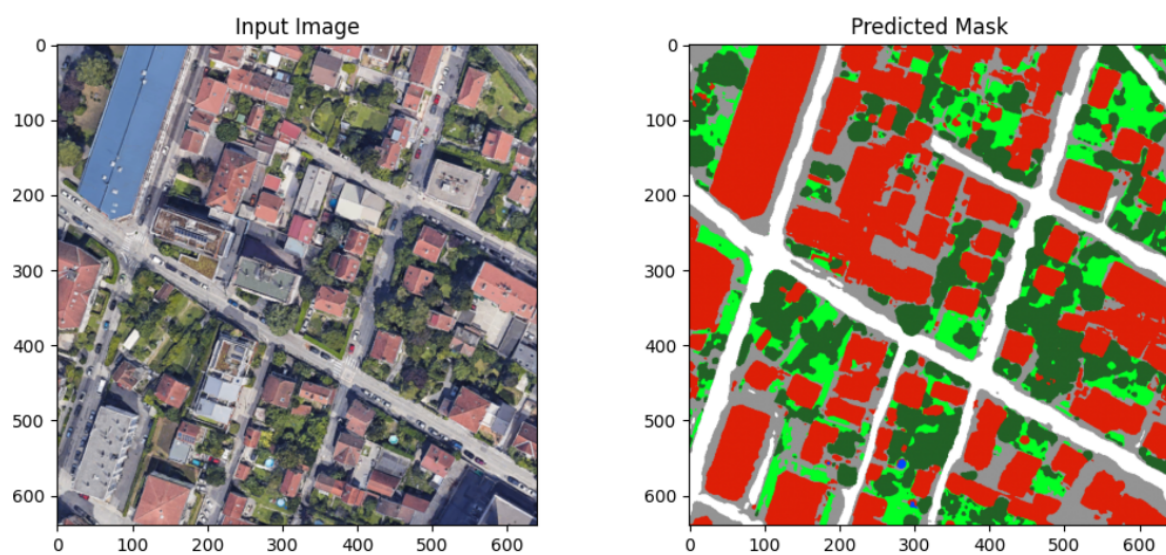


FIGURE 6 – Exemple de résultat 2

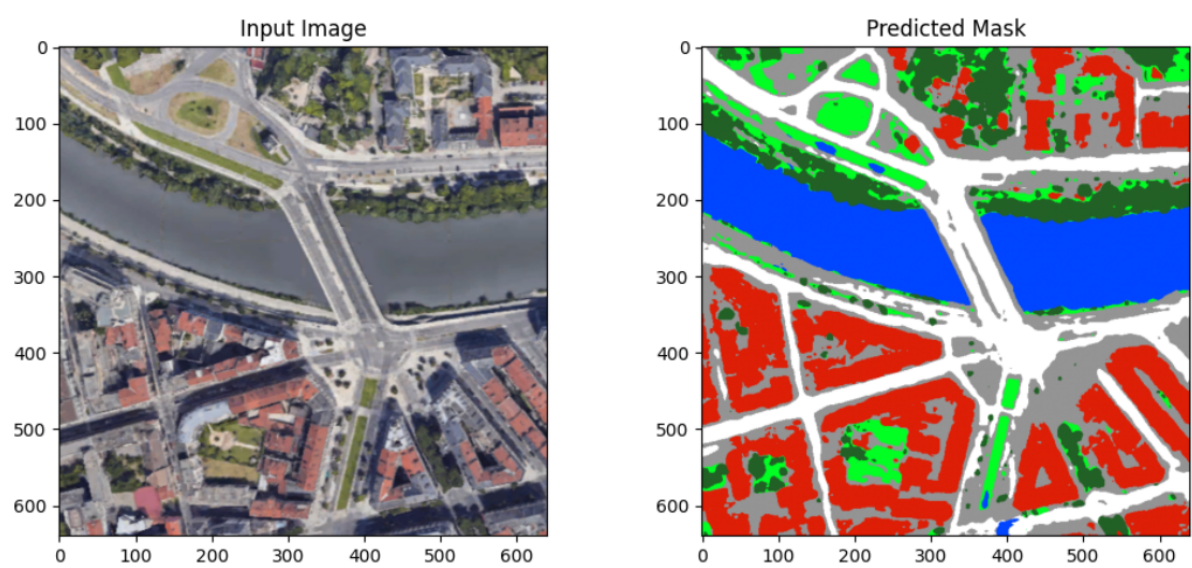


FIGURE 7 – Exemple de résultat 3

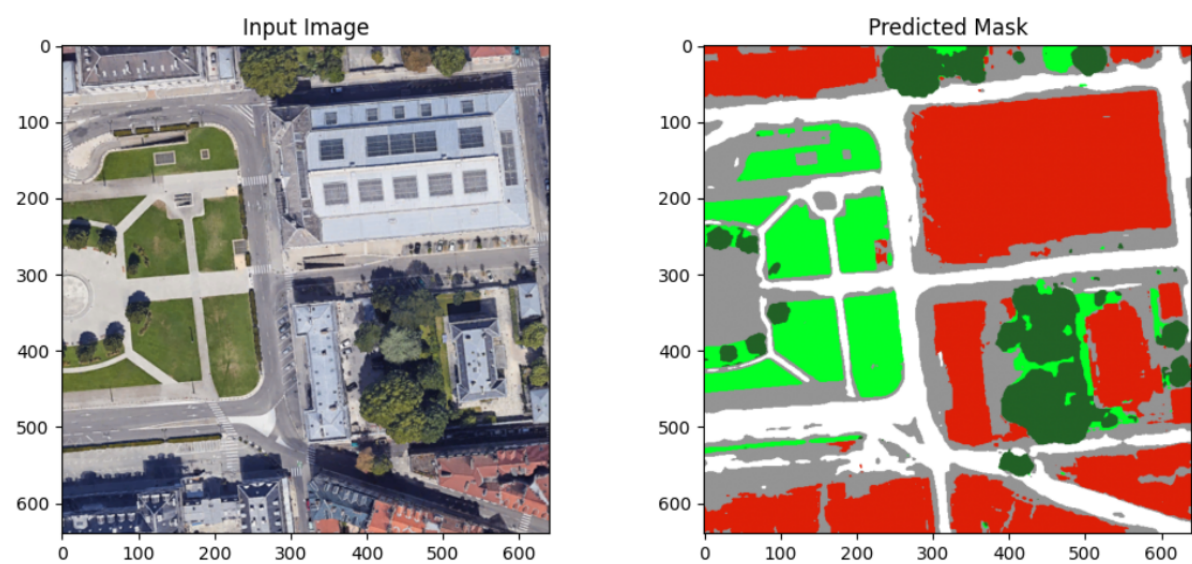


FIGURE 8 – Exemple de résultat 4

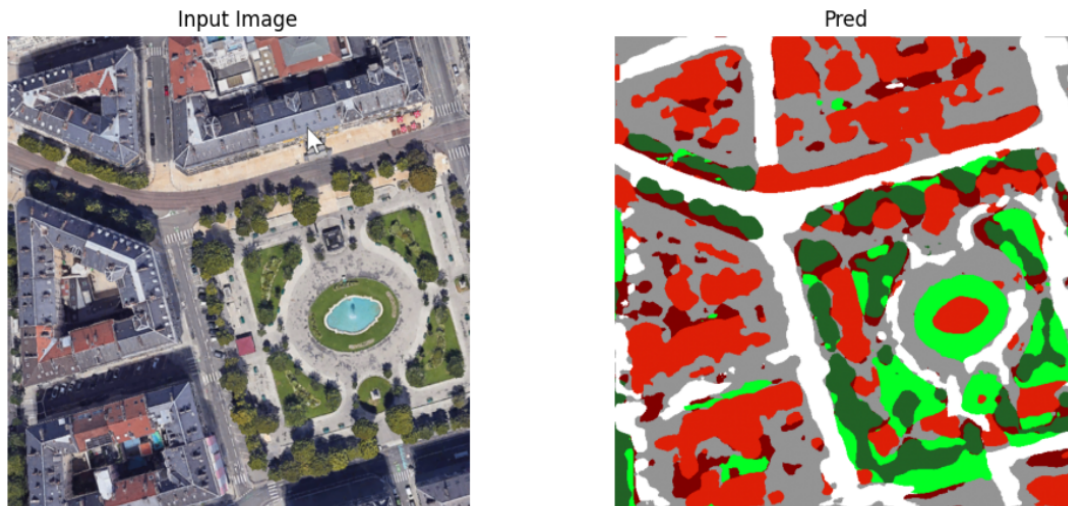
Les résultats obtenus grâce au modèle complet sont très pertinents. En effet, on remarque une très forte capacité de notre modèle à reconnaître de nombreux objets complexes tels que des bâtiments aux formes particulières où des environnements urbains très détaillés. Sur les figures précédentes, on observe une très bonne classification des différents objets (Rouge "bâtiments", Bleu "eau", vert clair "Espaces verts", vert foncé "Arbres", blanc "Routes", gris "Espace urbain"). On segmente idéalement la ville de Grenoble avec un faible taux d'erreur.

6.2 Résultat du modèle MOE

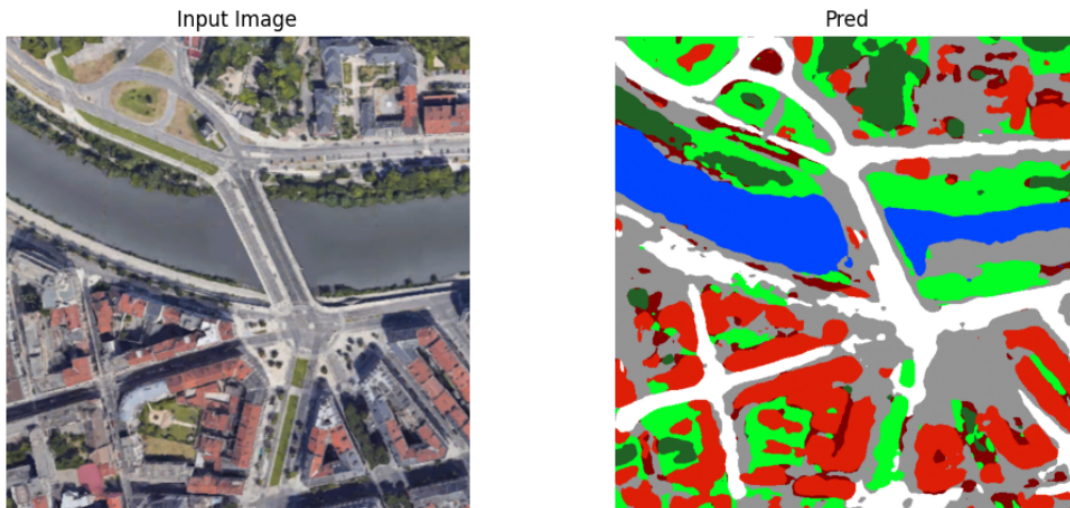
Pour le modèle avec des "experts" sur chaque classe, on obtient un modèle prédictif capable de classifier la ville de Grenoble. Le modèle a néanmoins quelques difficultés pour identifier des zones qui ne sont pas des objets clairs : routes, espaces développés,...

Ces difficultés viennent du fait qu'il n'est pas possible à notre échelle d'implémenter cette méthode avec le dataset des images des villes du monde entier (plus d'une heure par époque lors de l'entraînement). De ce fait, nous n'avons pu entraîner ce modèle que sur le bien trop petit dataset des images de Paris.

Voici un exemple des performances du MOE sur quelques images de l'agglomération grenobloise :



(a) Exemple de résultat MOE 1



(b) Exemple de résultat MOE 2

FIGURE 9 – Résultats MOE

On remarque que les résultats obtenus avec la stratégie de MOE sont plus pertinents que ceux obtenus avec un modèle complet à entraînement égal. En effet, les meilleurs modèles complets entraînés sur le jeu de données de la ville de Paris uniquement produisent des résultats moins cohérents : incapacité totale à reconnaître l'Isère, taux de bâtiments trop élevés. . . La stratégie de MOE permet d'éviter ces incohérences, d'une part car les experts sont meilleurs dans leurs domaines respectifs, d'autre part car nous pouvons seuiller les cartes de probabilités obtenues et accorder plus ou moins de crédit à des experts dont les taux de probabilités sont plus faibles mais les expertises meilleures.

7 Comparaison des résultats

Comme nous l'avons expliqué précédemment, nous sommes dans l'incapacité de mettre en place un modèle MOE entraînable sur l'ensemble de 2687 images à disposition de par le manque de temps et de ressources. Ayant eu une approche empirique sur l'ensemble du projet, comparer les couples (modèles/encodeurs) pour chaque expert sur un jeu aussi conséquent n'est pas possible à l'heure actuelle. Cependant, lorsqu'on confronte des résultats obtenus à entraînement égal, le MOE est plus convainquant. En effet, sa capacité à éliminer les petites erreurs et incohérences présentes sur les résultats d'un modèle complet nous permet d'obtenir une meilleure segmentation dans l'ensemble.

8 Conclusion

Pour conclure, ce projet nous a permis de mettre en application de nombreux savoirs et connaissances acquis en cours de Machine Learning et de Traitement de l'image. Nous avons eu une approche empirique sur l'ensemble des 40h de projet. En effet, nos résultats étant très visuels, nous avons pu valider/invalider nos hypothèses en affichant simplement nos résultats et en utilisant des indicateurs simples (`val_loss`, `IoU`...). De ce fait, nous avons pu comparer de nombreux couples modèles/encodeurs pour produire un algorithme complet idéal et pour entraîner par ailleurs des experts tous différents, certains encodeurs étant bien plus adapté à la reconnaissance de routes, d'autres à la reconnaissance de bâtiments... C'est cette approche empirique qui nous a permise de hiérarchiser des ensembles de combinaisons de manière à choisir les plus adaptés pour le modèle complet d'une part, chaque modèle du modèle MOE d'autre part. La construction complète du modèle MOE à partir des 2687 images reste donc l'ouverture principale de ce projet. En effet, ayant pu visualiser clairement le meilleur fonctionnement d'un modèle MOE par rapport à un modèle complet, nous sommes certain de l'obtention de résultats encore plus satisfaisants avec cette approche.

9 Sources

Le site d'OpenEarthMap d'où proviennent les images utilisées :

Contributors, M. O. J. T. A. B. (s. d.). OpenEarthMap. <https://open-earth-map.org/overview.html>

Documentation Python sur la bibliothèque OpenCV :

PythonTech. (s. d.). La bibliothèque openCV Python. <https://www.tresfacile.net/la-bibliotheque-opencv-python/>

Lien de téléchargement des données du Dataset :

Global Land cover Mapping - OpenEarthMap. (2024, 3 mai). Kaggle. <https://www.kaggle.com/datasets/land-cover-mapping-openearthmap>

Documentation Python sur la bibliothèque Pytorch :

Adam Stewart (University of Illinois at Urbana-Champaign), Caleb Robinson (Microsoft AI for Good Research Lab), Isaac Corley (University of Texas at San Antonio) & By Adam Stewart (University of Illinois at Urbana-Champaign), Caleb Robinson (Microsoft AI for Good Research Lab), Isaac Corley (University of Texas at San Antonio). (s. d.). Geospatial deep learning with TorchGeo – PyTorch. <https://pytorch.org/blog/geospatial-deep-learning-with-torchgeo/>

WV View. (s. d.). <https://wvview.org/>

Admin. (2023, 19 avril). Cartographier la végétation avec Segment Anything Model (SAM) et Resnet34. GEOAFRICA. <https://geoafrica.fr/cartographier-la-vegetation-avec-segment-anything-model-sam-et-resnet34/>

Sivakumar, D. (2023, 25 avril). Putting encoder - decoder together - scaler topics. Scaler Topics. <https://www.scaler.com/topics/nlp/encoder-and-decoder/>